ME5413 Report

Homework 3: Planning

MAI Jingyang

National University of Singapore

maij@u.nus.edu

A0304610X

College of Design and Engineering

Department of Mechanical Engineering

National University of Singapore

Session 2024/2025

I. Introduction

In the domain of mobile robot, finding a path/trajectory is usually required to be aligned to but not limited to the following criteria, reaching the goal, safe (collision-free), smooth (comfort), Kinematic/Dynamic constraints (feasible) and optimal (energy/time). As a subset, discrete planning is introduced to solve the realistic planning problem, which can be expressed as planning on a directed state transition graph. The A* (A-star) Algorithm is an informed search algorithm widely used for graph traversal and shortest path finding. In this assignment, a 1000x1000 figure is given, where the pixel with gray value 255 represents the free cell, while the one with gray value 0 represents the occupied. The title of each task is shown as follows,

- Task1: Global Planning with A* Algorithm.
- Task2: "Travelling Shopper" Problem.

In task1, we implement A* Algorithm successfully, and we also try 3 heuristic functions $h(x)$, Manhattan, Euclidean and Chebyshev. We also successfully degenerate A* algorithm to Dijkstra's Algorithm and Greedy Best First Search (GBFS) Algorithm by adjusting cost-so-far function $g(x)$ and heuristic functions $h(x)$. Through comparison, we found that using different $h(n)$ will slightly change the total number of cells visited. We observed that the shortest path obtained by degenerated Dijkstra is generally shorter than A*, but the total number of cells visited is slightly higher. The effect of GBFS is completely opposite, and the total number of cells visited is sharply reduced, which means that its computing speed is dramatically improved. In task2 we model the "Travelling Shopper" (TSP) problem as an optimal traversal problem on a 5-node weighted graph, applying Floyd's and Dijkstra's Algorithm to solve it with intuitive visualization, obtain exactly the same path and total distance 645.606m.

II. Methodology

A. *A* Algorithm*

A* Algorithm for the optimal path by evaluating each node using the function $f(n)$:

$$f(n) = g(n) + h(n),$$

where, n represents each point, cost-so-far function $g(x)$ is describing the distance cost from the start node to node n, and heuristic function $h(n)$ represents the cost from node n to the goal node. Three heuristic functions are used in task1, their formulas are shown as below, where D represents an adjustable variable to size the distance cost between two nodes:

- Manhattan $h(n)$:

$$h(n) = D * (|x_n - x_{end}| + |y_n + y_{end}|)$$

- Euclidean $h(n)$:

$$h(n) = D * \sqrt{(x_n - x_{end})^2 + (y_n - y_{end})^2}$$

- Chebyshev $h(n)$:

$$h(n) = D * \max\{|x_n - x_{end}|, |y_n - y_{end}|\}$$

By adjusting $g(x)$ and $h(x)$, A* Algorithm can be easily degenerated to Dijkstra's Algorithm and GBFS Algorithm. When $h(x)$ tends to 0, i.e., the cost from the current point to the goal point is ignored, A* Algorithm is degenerated into Dijkstra's Algorithm. When $g(x)$ tends to 0, i.e., the cost from the current point to the goal point is the only focus, A* Algorithm is degenerated into GBFS Algorithm.

The path finding process by A* Algorithm is illustrated as below.

```
Algorithm A*
1: Initialize an open list and a closed list.
      - Open list stores nodes that need to be processed.
      - Closed list stores nodes that have already been processed
2: current node <- start node and add current node to open list
3: while the open list is not empty do
      current node <- node with min{f(n)} in open list
```

```
        Remove current node from open list
        Add current node to closed list
        for each neighboring node of current node
            if neighbor node in closed list then
               Skip
            elseif neighbor node not in open list then
               Add neighbor node to open list
               current node <- its parent
               Calculate current node f(n), g(n), and h(n)
            elseif neighbor node in open list then
               Compute its new g(n)
               if new g(n) < previous g(n) then
                   Remove it from open list
                   Update its f(n), g(n), and h(n)
               end
            end
        end
 4: Repeat steps 3 until the goal node is found or the open list becomes empty.
```

B. *Graph Floyd's and Dijkstra's Algorithm for TSP*

In Task 2 we use a 5-node weighted graph to model this TSP problem. The 5 nodes are bidirectionally connected to the other 4 nodes, and the weight of each edge is calculated using the A* algorithm mentioned above. The structure of this graph can be seen as below. Please note that the weights of both sides of the same edge are not necessarily equal, which depends on the choice of the start point and the end point when using A* algorithm between two points. With this weighed graph, we apply Floyd's and Dijkstra's Algorithm to find the shortest path from node A, go through all nodes from node B to node E and then return back to node A.

From \ To	A	B	C	D	E
A	0	w_{BA}	w_{CA}	w_{DA}	w_{EA}
B	w_{AB}	0	w_{CB}	w_{DB}	w_{EB}
C	w_{AC}	w_{BC}	0	w_{DC}	w_{EC}
D	w_{AD}	w_{BD}	w_{CD}	0	w_{ED}
E	w_{AE}	w_{BE}	w_{CE}	w_{DE}	0

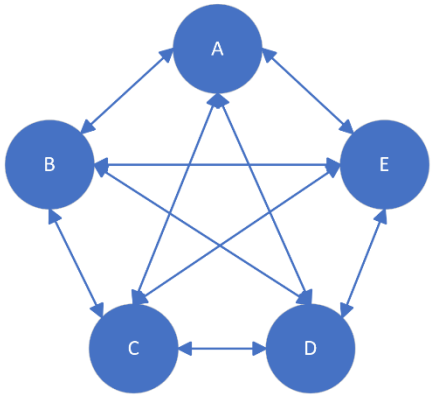

Figure 1 Weighted Graph Setting in Task2. Please note that the weights of both sides of the same edge are not necessarily equal.

III. Experiments and Results

A. *Task1: Global Planning with A* Algorithm*

In this section, we recorded the shortest path obtained by A* when finding path between two points after changing heuristic function to Manhattan $h(n)$ and Euclidean $h(n)$ in Table 1 and Table 2, as well as the total number of cells it visited. It is worth noting that the more cells it explored in total, the longer the computation time will be. We also used Chebyshev $h(n)$ and obtained similar results to the previous two, due to space limitations, we have not included the result of it below.

As we can see in Table 1 and Table 2, each shortest path length they obtain is exactly the same, but the total number of cells explored is slightly different. We observe that their computation times are generally similar.

Table 1 Effects Using Manhattan $h(n)$. The units in the table on the left are meters, and the units on the right are numbers. Table 2 to Table 3 below are following the same units.

From / To	start	snacks	store	movie	food
start	0	146.622	163.164	182.630	232.032
snacks	143.178	0	117.942	107.238	131.660
store	162.180	120.402	0	218.194	119.482
movie	180.334	109.534	223.934	0	115.168
food	233.836	130.676	114.398	113.856	0

From / To	start	snacks	store	movie	food
start	0	81573	80733	104319	152651
snacks	70073	0	60241	59658	82205
store	72973	54851	0	147590	61880
movie	94948	50033	159249	0	65350
food	165356	80985	62054	62361	0

Table 2 Effects Using Euclidean $h(n)$.

From / To	start	snacks	store	movie	food
start	0	146.622	163.164	182.630	232.032
snacks	143.178	0	117.942	107.238	131.660
store	162.180	120.402	0	218.194	119.482
movie	180.334	109.534	223.934	0	115.168
food	233.836	130.676	114.398	113.856	0

From / To	start	snacks	store	movie	food
start	0	98462	83212	106093	153257
snacks	72562	0	60834	61529	90348
store	76793	59488	0	157513	63617
movie	99294	53349	162507	0	67509
food	167227	90270	63523	62899	0

When it comes to degeneration, effects we degenerate A* to Dijkstra's Algorithm and GBFS are shown as follows. Table 3 shows that after degenerating to Dijkstra, the shortest path length of each pair of nodes is slightly shortened, but the total number of explored cells is slightly increased. This means that Dijkstra can obtain a shorter shortest path at the expense of more computational costs. Table 4 shows the amazing reduction in computational consumption of GBFS, where the total number of cells visited is significantly reduced compared to the original A*. At the same time, the shortest path it obtains is only slightly higher than the original A* overall.

Table 3 Effects degenerating A* to Dijkstra.

From / To	start	snacks	store	movie	food
start	0	140.882	159.064	178.202	224.324
snacks	140.882	0	114.170	106.418	129.528
store	159.064	114.170	0	208.354	112.594
movie	178.202	106.418	210.650	0	111.232
food	224.324	129.52	110.790	111.232	0

From / To	start	snacks	store	movie	food
start	0	147204	99589	125711	156807
snacks	86301	0	65952	66609	108049
store	93977	95949	0	168118	94342
movie	124734	81774	169843	0	84750
food	172374	129957	67968	68152	0

Table 4 Effects degenerating A* to GBFS.

From / To	start	snacks	store	movie	food
start	0	152.602	165.328	235.710	276.196
snacks	145.460	0	120.106	145.644	182.966
store	163.006	122.558	0	221.932	131.770
movie	210.974	141.392	254.404	0	118.702
food	253.020	132.780	116.024	115.272	0

From / To	start	snacks	store	movie	food
start	0	7361	7696	6477	12368
snacks	5295	0	586	1142	21259
store	10771	545	0	1845	1356
movie	3074	1787	2984	0	1336
food	9524	6141	702	550	0

B. *Task2: "Travelling Shopper" Problem*

In task2, we used A* with $h(n)$ as Manhattan in the process of calculating the weight of the graph. We used Floyd and Dijkstra to find the shortest path in the 5-node graph. The paths and lengths obtained by both methods are exactly the same. The paths are shown in the figure below, and their lengths are both 645.606m.

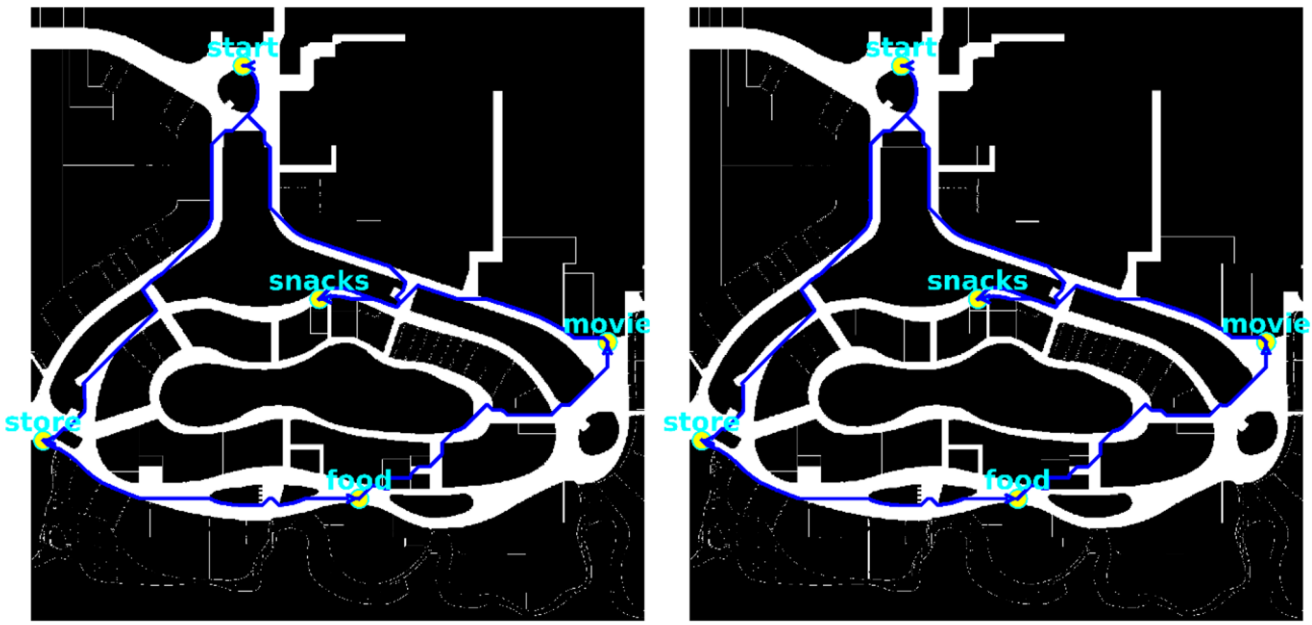

Figure 2 Shortest Path Illustration. The left one is the result of using Floyd's Algorithm, the right one is the result of Dijkstra.

IV. Discussion

In task 1, we implemented A* with different heuristics and observed that while Dijkstra's Algorithm found slightly shorter paths, it required visiting more nodes, increasing computational cost. In contrast, GBFS dramatically reduced visited nodes but at the expense of path optimality, highlighting its tendency to be misled by heuristics.

In task 2, we applied Floyd's and Dijkstra's Algorithms to solve the Travelling Shopper Problem (TSP) on a 5-node graph, obtaining the same optimal path and total distance of 645.606m. We do notice that the computational complexity of Floyd is $O(V^3)$ and the one of Dijkstra is $O(VlogV + E)$, where V is the number of nodes, E is the number of edges between node pairs. For large graphs, Floyd has too high computation consumption. For $E \ll V2$, i.e., the sparse graph, Dijkstra will have the computational advantage.

This work has limitations as follows. First, the number of $h(n)$ selected in this work is limited, and we have not selected in a much wider variety to analyze the impact of the choice of $h(n)$ on the performance of A* Algorithm. In addition, this work is only based on a relatively small environment for analysis and comparison. In the future, we should test it on more maps of different sizes to get a more comprehensive understanding.

V. Conclusion

In this assignment we successfully implement several planning algorithms as well as incorporate them with realistic data, visualize the results and evaluate them with comparison.